

Как писать самостоятельные APP на C и C++

Версия документа: 06.09.2026. Руководство для SDK ABI 3.

Здесь описан полный путь от `int main(void)` до файла `.APP`, который можно скопировать на MK61s и запустить. Приложение собирается отдельно от прошивки: её ELF, BIN и адреса функций для этого не нужны. Готовый APP работает на прошивках с совместимым API и включённым загрузчиком ABI 3.

Содержание:

- 1 [Модель приложения.](#)
- 2 [Первая программа и сборка.](#)
- 3 [Установка и запуск.](#)
- 4 [Сервисы прошивки.](#)
- 5 [Обработка файлов и графика.](#)
- 6 [Память, C++ и перенос существующего кода.](#)
- 7 [Сжатие, размеры и проверка.](#)

Модель приложения

APP — нативная программа для Cortex-M4 Thumb с аппаратным соглашением вызова `hard-float`, `fpv4-sp-d16`. Её машинный код и данные загружаются в общее окно SRAM размером 20 480 байт, начиная с `0x20000000`. Одновременно исполняется один APP. Вызов синхронный: возврат из `main()` завершает приложение и возвращает управление прошивке.

Возможности устройства предоставляет таблица функций `mk61_api`. Например, вместо `printf()` программа вызывает `mk61_api->display_write_utf8()`, а вместо платформенной клавиатуры использует `mk61_api->key_poll()`. SDK сам добавляет точку входа контейнера, подключает API и вызывает `main()`. Писать `mk61_module_entry`, `startup` или `linker script` не требуется.

Это freestanding C/C++: алгоритмы и небольшие библиотеки можно переносить, но программа для Windows/Linux с файловой системой, потоками и полным `libc` не заработает без адаптации. APP не является ELF-загрузчиком или средой ОС. Он содержит уже связанный образ, сжатый поток и 64-байтовый заголовок.

Штатные FOCAL, BASIC, WBMP, Markdown и CHIP-8 тоже используют ABI 3, но работают через отдельный [System API](#). Для нового пользовательского приложения достаточно обычного `mk61_app.h`.

Первая программа и сборка

Инструменты

Нужны исходники этого репозитория, Python 3.10+, ARM GCC и нативный C++17-компилятор для упаковщика. Проверен xPack ARM GCC 14.2.1 из STM32 Core. Для отдельного APP не нужны сборка Arduino-прошивки, CMake и Ninja.

Добавьте каталог `bin` ARM GCC в `PATH`. Если это неудобно, передавайте сборщику `--arm-toolchain-bin /путь/к/arm-gcc/bin`. Нативный компилятор запускается на компьютере и собирает упаковщик; ARM-компилятор создаёт код для калькулятора. Вместо сборки упаковщика можно указать готовый `--packer /путь/к/mk61_module_pack`.

Все дальнейшие команды выполняются из корня репозитория.

Исходник

Создайте каталог `my-app`, а в нём файл `main.c`:

```
#include "mk61_app.h"

int main(void) {
    const uint32_t required =
        MK61_APP_CAP_TEXT_DISPLAY | MK61_APP_CAP_KEYBOARD;
    if (!mk61_app_api_compatible(mk61_api, sizeof(*mk61_api), required))
        return MK61_APP_RUNTIME_ERROR;

    static const char message[] = "Hello, MK61s!";
    if (!mk61_api->display_clear() ||
        !mk61_api->display_write_utf8(0, 0, message, sizeof(message) - 1))
        return MK61_APP_RUNTIME_ERROR;

    for (;;) {
        const int32_t key = mk61_api->key_wait();
        if (key == MK61_APP_KEY_OK || key == MK61_APP_KEY_ESC)
            return MK61_APP_OK;
    }
}
```

Программа показывает строку и ждёт OK или ESC. `key_wait()` обслуживает фоновые задачи во время ожидания, поэтому отдельный цикл задержки не нужен. Символ завершающего NUL не включается в длину текста.

Команда сборки

macOS/Linux:

```
python3 tools/build_portable_app.py --name HELLO \
    --source my-app/main.c \
    --output-dir .build/my-app
```

Windows, PowerShell или cmd, одной строкой:

```
python tools/build_portable_app.py --name HELLO --source my-app/main.c --output-dir .build/
my-app
```

На Windows для автоматической сборки упаковщика нужен PowerShell 5.1+. Имя после `--name` состоит из 1–31 ASCII-символа: первая буква или цифра, далее также допустимы `_` и `-`. Расширение `.APP` добавляется автоматически. Зарезервированные имена вроде CON и NUL не принимаются.

Результат в `.build/my-app`:

Файл	Назначение
HELLO.APP	Устанавливаемый сжатый контейнер.
HELLO.elf	Связанный ARM ELF для анализа символов и дизассемблирования.
HELLO.map	Карта размещения кода и данных.
HELLO.bin	Несжатый образ приложения без BSS. Это не прошивка прибора.
HELLO.json	Размеры, выбранное сжатие и версия компилятора.

На устройство нужен только HELLO.APP. Не переименовывайте ELF или BIN в APP и не прошивайте HELLO.bin через DFU.

Этот исходник проверен сборкой как C и как C++: с GCC 14.2.1 в обоих случаях получился APP 189 байт, образ вместе с BSS — 132 байта. Стек в эту цифру не входит.

Несколько исходников и C++

`.c` собирается как C11, `.cpp` — как C++17. Тот же пример можно сохранить в `main.cpp` и заменить путь в `--source`. Можно смешивать языки; каждый исходник перечисляется отдельным `--source`, каталог заголовков — отдельным `--include`. Если C++ вызывает собственные функции из C-файла, оформите общий заголовок с `extern "C"` под `#ifdef __cplusplus`.

Готовый смешанный проект — [пример WBMP](#):

```
python3 tools/build_portable_app.py --name WBMP \
--source examples/portable-apps/WBMP/main.c \
--source examples/portable-apps/WBMP/viewer.cpp \
--source code/wbmp.cpp --handled-magic I1 \
--output-dir .build/my-wbmp
```

Для своего каталога заголовков добавьте `--include my-app/include`, для статической библиотеки — `--library my-app/lib/libalgo.a`. Библиотека должна быть собрана для того же Cortex-M4 Thumb, hard-float, fpv4-sp-d16, без зависимостей от отсутствующей ОС или libc. Объекты LTO требуют совместимого компилятора. Все необходимые реализации должны попасть в итоговую линковку: прямых импортов из ELF прошивки у обычного APP нет.

Установка и запуск

На устройстве должны быть включены загрузчик APP и `MK61_ENABLE_PORTABLE_APPS=1`. ABI 3 включён по умолчанию в комплектных F401-сборщиках GCC, Arduino CLI и плате Arduino IDE MK61s F401 + APP. Обычная F411-прошивка не включает APP; на F411 этот путь проверен в специальной сборке с загрузчиком и правильной раскладкой SRAM. Одного определения макроса в произвольном скетче недостаточно: нужен также linker script для окна APP.

Установите программу через менеджер `tools/mkc.cmd` либо USB-диск:

- 1 На калькуляторе откройте Меню → USB-диск.
- 2 На диске MK61S C5 создайте каталог Apps и скопируйте туда HELLO.APP.
- 3 Дождитесь окончания записи, безопасно извлеките диск на компьютере.
- 4 Нажмите ESC на калькуляторе, откройте Проводник и выберите APP клавишей OK.

Из USB-терминала эквивалентны команды:

```
open /Apps/HELLO.APP
run /Apps/HELLO.APP
```

Каталог /Apps — соглашение, а не требование загрузчика. Пользовательские APP можно перемещать и переименовывать; не занимайте канонические имена системных модулей в /System. При обновлении совместимой прошивки уже собранный ABI 3 не требует пересборки. Перед первым использованием ABI 3 старую прошивку с загрузчиком только ABI 2 нужно обновить.

Сервисы прошивки

Публичные объявления находятся в `mk61_app.h` и `loadable_app_api.h`. Версия контейнера ABI 3 и версия таблицы API 1 — разные номера. SDK проверяет magic, версию и размер таблицы. Программа дополнительно проверяет нужные возможности через `mk61_app_api_compatible()`, как в примере. Не копируйте структуру API в собственный заголовок и не меняйте её порядок.

Capability MK61_APP_CAP_...	Функции и поведение
TIME	<code>millis_ms()</code> , <code>service()</code> , <code>delay_ms(ms)</code> .
TEXT_DISPLAY	<code>display_columns()</code> , <code>display_rows()</code> , <code>display_clear()</code> , <code>display_write_utf8(col, row, text, bytes)</code> .
KEYBOARD	<code>key_poll()</code> возвращает событие либо <code>MK61_APP_KEY_NONE</code> ; <code>key_wait()</code> ждёт нажатия.
KEY_STATE	<code>key_pressed(key)</code> проверяет удержание клавиши, в том числе несколько клавиш одновременно.
LED	<code>led_set(enabled)</code> , <code>led_blink(count, on_ms, off_ms)</code> .
SOUND	<code>beep(hz, ms, volume_percent)</code> , <code>sound_stop()</code> .
FILES	<code>file_size(id)</code> , <code>file_read(id, offset, output, length)</code> . Только чтение.

Capability MK61_APP_CAP_...	Функции и поведение
GRAPHICS	Проверка экрана, начало, передача и завершение графического кадра; подробнее ниже.

У текстовых координат начало (0, 0). Узнавайте количество строк и столбцов через API. Одна запись принимает до MK61_APP_MAX_TEXT_BYTES (63 байт) валидного UTF-8 без NUL внутри строки; текущая реализация принимает символы BMP, до трёх байт UTF-8. Строка выводится в пределах оставшихся столбцов, автоматического переноса на следующую строку нет. Для длинного текста разбивайте его на строки и не разрезайте UTF-8-последовательность.

`display_clear()`, `display_write_utf8()`, `led_blink()`, `beep()` и операции начала/передачи графики возвращают 1 при успехе, 0 при отказе. Проверяйте результат: наличие capability не гарантирует успех операции. Это соглашение отличается от результата `main()` и файлового обработчика, где нулевой MK61_APP_OK означает успех.

Клавиши и длинные вычисления

Используйте MK61_APP_KEY_DIGIT_0...DIGIT_9, LEFT, RIGHT, OK, ESC и остальные имена из заголовка. Они обозначают логические клавиши, а их физическую раскладку выбирает прошивка. Для переносимого приложения не используйте аппаратные scan-code. Неизвестная клавиша кодируется как MK61_APP_KEY_RAW_BASE + scan_code.

`key_poll()` выдаёт события нажатия, а не отпускания. Для удержания нужен `key_pressed()`. Оба вызова обслуживают фоновые задачи; `key_wait()` делает это в своём цикле ожидания.

Во время длительных вычислений разбивайте работу на небольшие порции и регулярно вызывайте `service()`. Он обслуживает прошивку, но не завершает за вас приложение по ESC: обработку выхода нужно написать явно. Вместо пустых циклов задержки используйте `delay_ms()`. Для интервалов корректна беззнаковая разность `(uint32_t)(mk61_api->millis_ms() - started_at)`: счётчик миллисекунд переполняется. Перед использованием времени проверьте MK61_APP_CAP_TIME.

Обработка файлов и графика

APP как просмотрщик файла

Обычный запуск вызывает `main()`. Чтобы приложение также открывало файлы, реализуйте необязательную функцию:

```
uint32_t mk61_app_open_file(uint32_t file_id);
```

В C++ определение можно явно оформить как `extern "C" uint32_t mk61_app_open_file(uint32_t file_id)`. Декларация уже есть в `mk61_app.h`. Без собственной реализации SDK возвращает MK61_APP_INVALID_FILE.

При сборке задайте `--handled-magic I1` для WBMP, T2 для Markdown либо другой поддерживаемый тип C5. Это двухсимвольная метка типа файла в C5, а не первые байты его содержимого. Параметр не регистрирует произвольное новое расширение: соответствие имени файла типу должно знать само C5.

При открытии файла прошивка передаёт обработчику `file_id`. Для чтения не нужны путь и `fopen()`:

Вызов	Результат
<code>file_size(file_id)</code>	Размер в байтах; UINT32_MAX при ошибке. Ноль означает пустой файл.
<code>file_read(file_id, offset, buffer, length)</code>	Число прочитанных байт; UINT32_MAX при ошибке. Для точного чтения сравнивайте результат с <code>length</code> .

Текущая реализация принимает ID, смещение и длину в диапазоне 0...65535. Файлы удобно читать порциями; проверяйте длину до разбора заголовка и не доверяйте размерам из содержимого. Обычный API не предоставляет создания, записи и удаления файлов, обхода каталога или открытия произвольного пути.

Обработчик возвращает один из кодов MK61_APP_OK, MK61_APP_INVALID_FILE, MK61_APP_UNSUPPORTED_DISPLAY, MK61_APP_BUSY, MK61_APP_IO_ERROR, MK61_APP_RUNTIME_ERROR.

`main()` также возвращает ноль при успехе; ненулевой результат означает ошибку запуска, а не число для регистра X.

Встроенный обработчик и канонический System APP имеют приоритет перед пользовательским APP. Для проверки примера WBMP нужен комплект без штатных обработчиков WBMP/Markdown, иначе файл откроют они. Два пользовательских APP с одной сигнатурой создают неоднозначность. При прямом запуске примера WBMP .APP выводится подсказка; просмотр начинается при открытии самого .wbmp.

Графический экран

После проверки `MK61_APP_CAP_GRAPHICS` вызовите `graphics_available()`: графический код может быть собран, но текущий экран окажется символьным. Далее выполните `graphics_begin()` и проверьте его результат. Размер кадра узнаётся через `graphics_width()` и `graphics_height()`. Обычный APP API сейчас предоставляет кадр 192×64, то есть 1536 байт, для UC1609 или совместимого USB-экрана. Возможности внутреннего System API для других дисплеев не следует автоматически переносить на этот контракт.

Байты расположены полосами высотой восемь пикселей, младший бит сверху:

```
/* frame обнулён, 0 <= x < width, 0 <= y < height. */
frame[(y / 8) * width + x] |= (uint8_t)(1U << (y % 8));
```

Передавайте весь кадр через `graphics_present(frame, frame_bytes)`. Между кадрами обслуживайте события. При изменении `graphics_revision()` завершите сеанс, заново проверьте экран и откройте графику ещё раз. После успешного `graphics_begin()` вызывайте `graphics_end()` на каждом пути выхода, включая ошибки. Полный пример такого цикла находится в [WBMP/viewer.cpp](#).

Память, C++ и перенос существующего кода

В 20-КиБ окно входят машинный код, константы, начальные значения .data, BSS и выравнивание. Сжатие уменьшает файл, но не это потребление RAM. Большой обнулённый массив почти не увеличивает APP на диске, однако целиком занимает BSS после загрузки.

Стек общий с прошивкой и находится за пределами окна APP. Не считайте всю свободную SRAM личным стеком приложения. Избегайте больших локальных массивов и глубокой рекурсии; буферы с известным сроком жизни можно сделать статическими, учитывая лимит BSS. Обычному приложению не выдаются внутренние workspace-арены System APP.

При каждом запуске пользовательского APPLICATION ABI 3, включая вызов обработчика файла, .data восстанавливается из образа, а BSS обнуляется. Например, `static unsigned counter;` снова равен нулю при следующем открытии APP. На сохранение состояния между запусками рассчитывать нельзя. У System APP другой режим кэширования, описанный в System API.

Поддерживаются структуры, классы и шаблоны, не требующие отсутствующего runtime. Допустимы константы и глобальные данные со статической инициализацией. Глобальные конструкторы/деструкторы запрещены линкером. Исключения, RTTI и поддержка потокобезопасной инициализации static отключены. Не используйте `std::string`, контейнеры с динамической памятью, `iostream`, `new/delete`, `malloc/free` без собственной подходящей реализации.

SDK добавляет `memcpy`, `memmove`, `memset`, `memcmp` и необходимые арифметические функции `libgcc`. Полный `libc` и `libm` автоматически не подключаются: даже доступный при компиляции заголовок не означает, что реализация `printf`, `strlen`, `sin` или `fopen` войдёт в APP. При переносе небольшую необходимую функцию можно включить исходником или ARM-библиотекой, после чего проверить размер и отсутствие внешних зависимостей.

Практический порядок переноса:

- 1 Отделите алгоритм от ввода-вывода и платформенных функций.
- 2 Замените экран, клавиши, время и чтение файла вызовами `mk61_api`.
- 3 Уберите зависимость от ОС, динамических глобальных объектов и большого heap.
- 4 Добавьте `main()` и при необходимости `mk61_app_open_file()`.
- 5 Перечислите исходники в команде сборки, проверьте `memory_bytes` и ошибки линкера.
- 6 Проверьте повторный запуск, выход и обработку неподдерживаемого экрана.

При переносе прежнего пользовательского ABI 2 вместо собственного `mk61_module_entry` используйте `startup SDK`; уберите `resident ELF`, `--just-symbols` и `manifest`-сборку. Вызовы прежнего C++ API заменяются эквивалентными полями `mk61_api` и именами `MK61_APP_...` из C-заголовка. Состояние, которое раньше сохранялось в кэше APP, теперь нужно считать локальным одному запуску. Системные модули переносились отдельно через System API; подключение их адаптеров не входит в обычную сборку с `--source`.

Сжатие, размеры и проверка

Сборщик сам связывает ELF, извлекает образ и упаковывает APP. Для ABI 3 проверяются два варианта: ZX0 и ARM Thumb BCJ → ZX0. BCJ обратимо преобразует команды перехода перед сжатием. Выбирается меньший поток; при равенстве — обычный ZX0. Приложению не нужны код распаковщика, ручная настройка фильтра или дополнительный буфер распаковки.

В отчёте `NAME.json`:

Поле	Что измеряет
<code>image_bytes</code>	Несжатый образ кода и начальных данных.
<code>bss_bytes</code>	Обнуляемый хвост памяти, включая выравнивание.
<code>memory_bytes</code>	<code>image_bytes</code> + <code>bss_bytes</code> , максимум 20 480.
<code>app_bytes</code>	Сжатый поток плюс заголовок 64 байта, максимум 20 544.
<code>compression</code>	ZX0 или ZX0+BCJ; последнее означает BCJ перед ZX0.
<code>compiler</code>	Версия компилятора для воспроизведения сборки.

Размер APP-файла не включает метаданные C5. Стек, внутренние буферы прошивки и System workspace не входят в `memory_bytes`. Сравнение штатных модулей до и после перехода приведено в [отдельном отчёте](#).

Ошибки сборки и запуска

Сообщение или симптом	Что проверить
specify --arm-toolchain-bin...	ARM GCC отсутствует в PATH; укажите каталог его bin.
undefined reference	Не перечислен исходник/библиотека или вызвана функция вне SDK.
unsupported global constructors/destructors	Есть глобальный C++-объект с динамической инициализацией или деструктором.
MK61 module does not fit the SRAM overlay	Код + данные + BSS превысили 20 КиБ; уменьшайте их, а не только сжатый файл.
unpacked ELF section	Исходник или библиотека создали секцию, не поддерживаемую linker script.
PUT_APP / corrupt app	Повреждён контейнер либо прошивка не поддерживает его формат; проверьте сборку и копирование.
PUT_FIRMWARE / app/firmware mismatch	Для ABI 2 проверьте точную прошивку. Для ABI 3 — поддержку загрузчика, раскладку SRAM и требования API.
Вместо своего просмотрщика открылся штатный	Сработал приоритет встроенного/System-обработчика.
ГРАФИКА / НЕДОСТУПНА	Активный дисплей не предоставляет нужный графический режим.
Кажется, что калькулятор завис	Проверьте вызовы <code>service()</code> , обработку выхода и расход стека.

Для проверки контейнера и SDK на компьютере:

```
MK61_TEST_SANITIZERS=1 bash tests/run_portable_app_tests.sh
```

Этот набор проверяет SDK и упаковку, а не произвольную новую программу. Свой алгоритм удобно отдельно проверять host-тестами с подменой таблицы API; пример для просмотра WBMP — [portable_wbmp_self_test.cpp](#). На устройстве проверьте первый и повторный запуск, возврат через ESC, пустые и повреждённые входные файлы, смену экрана во время работы и восстановление обычного интерфейса после выхода.

Для воспроизводимого выпуска сохраните исходники, команду сборки, версию репозитория/SKD, версию toolchain и итоговые .APP/.json/.map. У ABI 3 не нужно прикладывать точный resident ELF для повторной сборки; требования к API и экрану приложения всё же следует указать.

Технический формат контейнера описан в [sdk/portable/README.md](#), системный интерфейс — в [SYSTEM-API.md](#), а прежняя manifest-сборка ABI 2 — в [руководстве по ABI 2](#).